management, and once proven, then the option to migrate to a native contract testing tool could be an easier transition.

We've chosen Pact as the preferred tool for the book due to its multilanguage support and high level of community support; we're also most familiar with Pact. However, writing contract tests without a tool is possible (see an example from one of the Pact authors at https://github.com/DiUS/diy-contract-example) if you want to take and implement the theory independently, but using a tool like Pact will make the process easier.

See appendix A, sections A.1 and A.2, for a brief description and review of the leading contract testing tools available. To conclude our findings on the tooling, there are a few different variants:

- Native contract testing tools such as Pact and Spring Cloud Contract (and probably others not mentioned here)
- Adapted integration testing tools such as Postman (and probably others not mentioned here)

3.7 Contract Testing Approaches

At the beginning of this chapter, we mentioned that there are two main approaches when it comes to contract testing:

- The traditional consumer-driven contract testing (CDCT) approach
- The provider-driven contract testing (PDCT) approach

Let's take a look at these two approaches in detail.

3.7.1 Consumer-driven contract testing

Ian Robinson first posted about the consumer-driven contract pattern in 2006 (https://mng.bz/V205). Ian described the challenges with schema versioning and breaking changes, showing the need for this style of testing more than 17 years ago. But still, CDCT isn't part of every testing strategy or on the radar for most projects. There is nothing much to say about